

KUBERNETES

Introduzione a Kubernetes

Kubernetes è un software sviluppato originariamente da **Google nel 2014** che permette di **gestire e orchestrare architetture basate sul concetto di container**. Presuppone quindi di conoscere bene i container e abbastanza bene Docker.

Il salto rispetto a Docker Compose. Con Compose si riescono già ad avviare più applicazioni basate su container. Ma nel momento in cui si vogliono creare **architetture molto più grandi con carichi di lavoro molto più importanti**, bisogna passare a Kubernetes. È esattamente il punto in cui si era chiusa la sezione su Docker.

Compose **compone** pochi container su una macchina. Kubernetes **orchestra** tanti container su tante macchine.

La prova del suo successo è la diffusione sul cloud: AWS, Google Cloud Platform e Azure lo supportano tutte. Il corso usa il **Kubernetes Engine di Google Cloud (GKE)**, ma i concetti valgono per quasi tutti i provider — cambiano fondamentalmente i comandi di accesso.

L'architettura minima: due soli concetti.

- **Nodo** — lo si immagina come una macchina, un computer, una macchina virtuale. È il posto dove girano i container.
- **Cluster** — un insieme di nodi.

I nodi si dividono in **master** (coordinano gli slave, uno per cluster) e **slave/worker** (contengono la nostra applicazione, i nostri container, e sono la maggioranza).

Il setup. Si crea un account su Google Cloud. Google fornisce **300 \$ di credito gratuito da spendere nei primi 90 giorni** — i prodotti cloud si pagano, quindi questo credito serve per fare le prove. Viene chiesta una carta di credito, ma **non viene addebitato nulla**: serve solo a verificare che non si sia un robot.

Dettaglio pratico che il corso non dice: un cluster GKE lasciato acceso **consuma credito anche se non lo usi**. Cancellalo quando finisci di provare.

Creare il cluster. Menu → Kubernetes Engine → Cluster → Crea. Si dà un nome (prova-`cluster-1`), si lascia la regione di default, si lascia il networking di default, si clicca Crea. Si usa il **wizard**, quindi configurazioni basiche senza impostazioni complicate. Ci vogliono **alcuni minuti**. Nell'esempio il cluster ha **un solo nodo**, che quindi fa sia da master che da slave.

Collegarsi. Si può lavorare con la UI o con la Shell; il docente preferisce la Shell. Attiva Cloud Shell (meglio aprirla in una finestra a parte), poi sul cluster il pulsante **Connect** fornisce già pronto il comando da copiare e incollare:

```
gcloud container clusters get-credentials <nome-cluster> --zone <zona> --project <id-progetto>
```

La sigla con i numeri nella stringa è l'**ID del progetto** Google Cloud. Verifica del client con `kubectl version`.

Il primo deploy, due comandi. L'immagine è quella già pubblicata dal docente su Docker Hub: contiene un servizio REST che ritorna una stringa.

```
kubectcl create deployment docker-springboot-helloworld-2 \
--image=programmingacademydotit/docker-springboot-helloworld-2:0.0.1-SNAPSHOT
```

```
kubectcl expose deployment docker-springboot-helloworld-2 --type=LoadBalancer --port=8081
```

Verifica. Servizi e ingressi mostra l'applicazione, lo stato, il tipo (load balancer) e l'**endpoint pubblico**. Cliccandoci sopra e aggiungendo /esempio:

<http://34.27.244.111:8081/esempio>

L'IP esterno lo assegna il load balancer di Google e **può metterci qualche minuto** a comparire.

Il senso della lezione: in due minuti e due comandi hai un container esposto al mondo. E con Kubernetes gestisci **numero di istanze, load balancing e tolleranza ai guasti** in modo molto semplificato.

Pod

Kubernetes ha l'obiettivo di distribuire i container sui worker node. Però **non li distribuisce direttamente sui nodi**: li **incapsula dentro oggetti chiamati pod**.

Il pod è l'**oggetto più piccolo che si può creare in Kubernetes** e rappresenta **la singola istanza di un'applicazione**.

Nell'esempio della slide: due nodi, quattro pod distribuiti equamente, un container per pod.

Lo scaling orizzontale. Parti da un pod solo sul worker node 1. A un certo punto gli utenti aumentano in maniera esponenziale: si **scala orizzontalmente**, cioè si distribuisce il carico tra più istanze — si **aumenta il numero di pod**. Se i due nodi non reggono più, si aggiunge un nodo.

★ **La regola da non sbagliare mai**: per aumentare le repliche si aumenta il **numero di pod**, non il numero di container dentro lo stesso pod.

Un pod = un container? Quasi sempre sì. Ma esistono due container ausiliari:

	Quando gira	A cosa serve
Init container	<i>prima</i> del container principale	script o utility da eseguire prima dell'avvio dell'app
Sidecar container	<i>in parallelo</i> al container principale	ne supporta il funzionamento: monitoraggio, metriche, raccolta log

I comandi.

```
kubectcl get pod -o wide      # tutti i pod: nodo, IP, stato
```

```
kubectcl describe pod <nome-pod> # dettaglio di UN pod
```

La differenza che il docente ripete: il primo mostra le informazioni di **tutti** i pod, il secondo descrive **il singolo** pod.

Cosa si legge nel `describe`:

- **Name** — il nome del pod.
- **Namespace** — qui è rimasto `default` per semplicità, ma è importante perché **si suddividono le risorse assegnandole ai namespace**. Serve per creare più ambienti: sviluppo, test, ecc.
- **Labels** — etichette che identificano il pod. Saranno **molto importanti** con i ReplicaSet e nei file YAML.
- **Annotations** — informazioni **non identificative**, che **Kubernetes non usa direttamente**: ID della build, autore della build.
- **Containers** — qui si legge il **nome del container**, che servirà nella lezione 4.

L'importanza del pod: mette insieme più container, **dà loro un IP** e li categorizza con le label.

Corollario: i container dello stesso pod si parlano su `localhost`, e gli IP dei pod sono **effimeri** — il che è esattamente il problema che risolve il Service.

ReplicaSet

Il ReplicaSet è un oggetto che permette l'avvio e la gestione di **più pod in replica**.

Una volta deciso il numero di pod da avviare, il ReplicaSet **garantisce che quel numero sia sempre in esecuzione**. Se succede qualcosa, li riavvia.

```
kubectl get replicaset
```

```
kubectl get rs      # forma abbreviata
```

L'output dà nome, **DESIRED** (quanti ne vogliamo), **CURRENT**, **READY**.

La dimostrazione che funziona davvero. Il docente prende l'ID di un pod e lo ammazza:

```
kubectl get pod -o wide
```

```
kubectl delete pod <id-del-pod>
```

```
kubectl get rs
```

Il pod viene effettivamente cancellato, ma **ne riparte subito un altro**: il numero resta quello impostato.

Scalare. Per passare a tre pod:

```
kubectl scale deployment docker-springboot-helloworld-2 --replicas=3
```

Subito dopo, `kubectl get rs` mostra **desiderati 3, correnti 3, pronti 1**: si stanno avviando.

Dopo qualche secondo sono tutti e tre pronti — è normale, serve il tempo di avvio dell'applicazione.

Precisazione importante: nella pratica il ReplicaSet **non si crea e non si tocca a mano**. Lo crea e lo gestisce il **Deployment** — infatti anche qui lo `scale` viene dato

sul deployment. Se lo scali direttamente, al primo aggiornamento del deployment la modifica si perde.

Deployment – aggiornare l'immagine

Lo scenario. L'app è già deployata. Hai la versione 1 e vuoi passare alla 2.

```
kubectl set image deployment/<nome-deployment> <nome-container>=<immagine>:<tag>
```

Servono **due nomi diversi** — nome del deployment e nome del container — ed è qui che ci si incarta. Se non li ricordi:

```
kubectl get deployment      # nome del deployment
```

```
kubectl get pod             # elenco dei pod
```

```
kubectl describe pod <id>    # sotto "Containers": il nome del container
```

L'esperimento chiave: l'immagine sbagliata. Il docente dà **di proposito** un'immagine inesistente:

```
kubectl set image deployment/docker-springboot-helloworld-2 \
  docker-springboot-helloworld-2=prova-img-errata:test
```

Cosa succede:

- **il servizio continua a funzionare** — l'endpoint risponde ancora;
- `kubectl get pod` mostra un pod in più che **non è partito**, stato **InvalidImageName**, `READY 0/1`;
- `kubectl describe pod <id>` nella sezione **Events** mostra il **Warning**: non riesce a scaricare l'immagine, non esiste.

★ **La lezione da portarsi a casa:** se c'è un errore durante il deployment, **la versione già installata rimane su**. Kubernetes verifica che il nuovo pod sia pronto prima di sostituire il vecchio. È il *rolling update*, configurato dai parametri che vedrai nella lezione 7.

Poi rifà il comando con l'immagine giusta (`0.0.3-SNAPSHOT`) e il deploy va a buon fine.

Dettaglio pratico: quando incolli il comando nella Cloud Shell **deve stare tutto su un'unica riga**.

Due comandi che il corso non mostra ma che completano il quadro: `kubectl rollout status deployment/<nome>` per seguire l'aggiornamento e `kubectl rollout undo deployment/<nome>` per tornare indietro.

Service

Un Service è **un'astrazione che definisce un insieme logico di pod e una politica per accedervi**.

Il perché esiste: i pod hanno un IP, ma **questi IP sono effimeri** — cambiano per scalabilità, guasti, riavvii. Il Service permette di accedere ai pod **in modo affidabile**.

La dimostrazione. Tre pod in Running, ognuno con il suo IP. Il docente ne killa uno:

```
kubectl delete pod <id-del-pod>
```

Risultato: viene creato **un nuovo pod con un altro IP** (grazie al ReplicaSet), e **nel frattempo il service continua a rispondere come se nulla fosse**. L'indirizzo esterno **rimane invariato**.

★ È il punto più importante della lezione: **gli IP dei pod ballano, l'indirizzo del Service no.**

Quando è stato creato il Service? Senza dirlo, già nella prima lezione — **con expose**:

```
kubectl expose deployment <nome> --type=LoadBalancer --port=8081
```

Dalla UI si vede come **bilanciatore del carico esterno**: espone i pod all'esterno e **dà in automatico il load balancing**, servizio fornito direttamente dal provider cloud.

I tipi. `kubectl get services` mostra due servizi:

Tipo	Cosa fa	Indirizzo
LoadBalancer	espone il gruppo di pod all'esterno , con LB del cloud provider	ha un EXTERNAL-IP
ClusterIP	IP interno al cluster , accessibile solo dall'interno	non ha IP esterno

Il dettaglio da notare nell'output: alla riga del ClusterIP **manca l'indirizzo esterno**.

Terzo tipo che il corso non nomina: **NodePort**. LoadBalancer di fatto ci si costruisce sopra — ed è il motivo per cui nel YAML del service (lezione 7) compare un campo `nodePort` che il docente cancella.

Il docente chiude ricordando che **questo è un corso introduttivo**: Kubernetes è molto complesso, soprattutto nella gestione dei casi particolari.

Tipo di nodi

Due tipi: **Master Node** e **Worker Node**. La maggior parte sono worker; **in minima parte** ci sono uno o più master che controllano l'intero cluster.

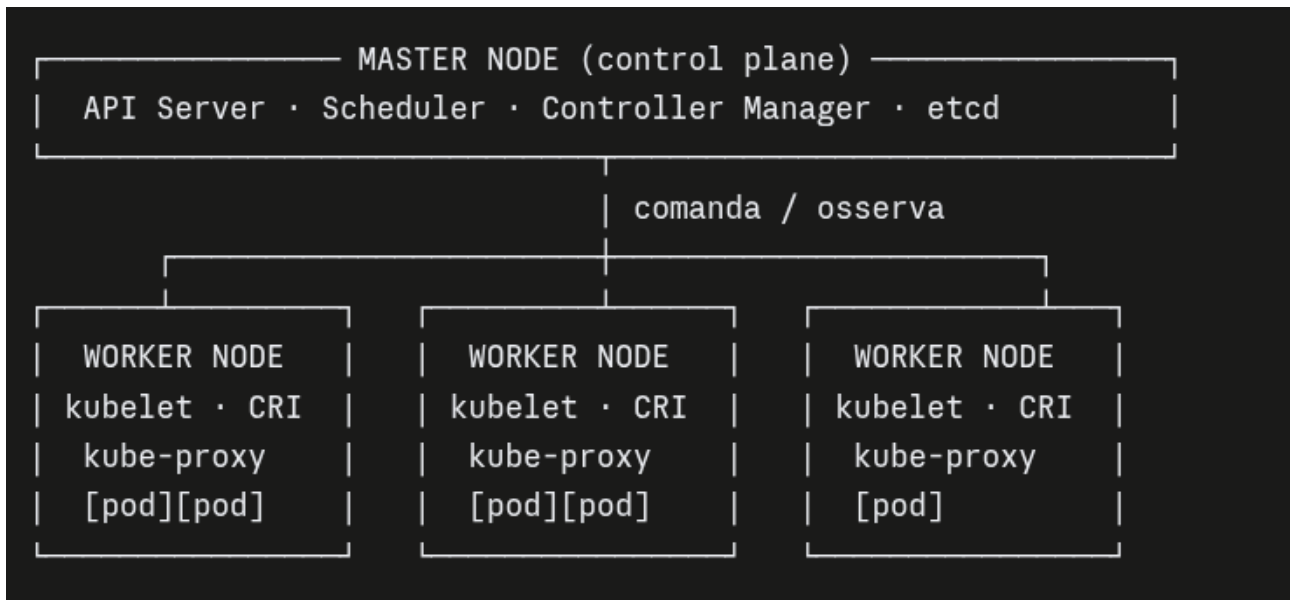
Master node — ospita **il database di Kubernetes e i processi fondamentali**:

Componente	Cosa fa
API Server	invocato ogni volta che si vuole modificare qualcosa nel cluster
Scheduler	schedula i pod , decide su quale nodo farli girare
Controller Manager	controlla la salute di tutto il cluster
etcd	database chiave-valore con le informazioni sul cluster (es. numero di istanze)

Ogni `kubectl` che scrivi finisce all'**API Server**. È l'unico punto d'ingresso: niente cambia senza passare da lì.

Worker node — è il nodo lavoratore, ci girano i container. Non devono per forza essere container Docker: **l'importante è che siano compatibili con Kubernetes**.

Componente	Cosa fa
kubelet	lavora in stretta collaborazione con il control plane per garantire che i pod girino in modo affidabile e rispettino risorse e configurazioni
CRI	responsabile dell' avvio e della gestione dei container
kube-proxy	gestisce le regole di rete del nodo , dentro e fuori dal cluster



Note di attualità, non del corso: oggi si dice **control plane** e **worker node** invece di master/slave; su un cluster gestito (GKE, EKS, AKS) il **control plane lo gestisce il provider** — ecco perché nel corso non ci si entra mai; e il runtime standard è **containerd**, dato che Docker come runtime è stato rimosso da Kubernetes nella 1.24 (le immagini Docker restano validissime).

Analisi del YAML

Farsi dare il YAML invece di scriverlo. Il modo comodo di partire:

```
kubectl get deployment <nome> -o yaml
```

```
kubectl get service <nome> -o yaml
```

Attenzione all'ordine: è **-o yaml**, non **yaml -o**. Il docente sbaglia e si corregge in diretta.

La pulizia. L'output è pieno di **metadati aggiunti da Kubernetes e da Google Cloud** che vanno tolti: `creationTimestamp`, `generation`, `resourceVersion`, `uid`, le annotation del provider, `revisionHistoryLimit`, il timestamp dentro `template`, `resources`, `dnsPolicy`, e tutta la sezione **status**. `restartPolicy` invece si lascia.

Regola semplice: **tutto ciò che Kubernetes genera va via, tutto ciò che hai deciso tu resta.**

Il deployment ripulito:

apiVersion: apps/v1

kind: Deployment

metadata:

name: k8s-applicazione-2

namespace: **default**

labels:

app: k8s-applicazione-2

spec:

replicas: 1

selector:

matchLabels:

app: k8s-applicazione-2

strategy:

type: RollingUpdate

rollingUpdate:

maxSurge: 25%

maxUnavailable: 25%

template:

metadata:

labels:

app: k8s-applicazione-2

spec:

containers:

- name: k8s-applicazione-2

image: programmingacademydotit/k8s-applicazione-2:0.0.4-SNAPSHOT

imagePullPolicy: IfNotPresent

restartPolicy: Always

- **metadata** — nome del deployment, namespace, label.
- **replicas** — quanti pod creare. Stesso concetto di `kubectl scale`.
- **selector.matchLabels** — *tutto quello che definiamo qui va applicato al pod con questa label. Deve corrispondere esattamente* alla label dentro

`template.metadata.labels`: è il filo che lega il Deployment ai suoi pod. ★ È la fonte di errore numero uno.

- **template** — la definizione del pod: le label che deve avere e quali immagini deve contenere.
- **imagePullPolicy: IfNotPresent** — *se l'immagine non è già presente scaricala, altrimenti no.*
- **restartPolicy: Always** — se succede qualcosa, riavvia.
- Per **più container nello stesso pod** basta ripetere il blocco sotto `containers`, ma il docente avverte che **è buona norma tenerne uno solo**.

strategy — come si passa da una versione all'altra:

Campo	Significato
maxSurge: 25%	quanti pod in più al massimo durante il passaggio. Con 4 pod: al massimo 1 in più
maxUnavailable: 25%	quanti pod possono non essere disponibili durante il deployment. Con 4 pod: al massimo 1

Il senso è il rolling update: si passa da una versione all'altra **in modo meno traumatico**.

«Immaginate questa roba in produzione: non è che potete bloccare il sito di una banca.»

Il service ripulito:

`apiVersion`: v1

`kind`: Service

`metadata`:

name: k8s-applicazione-2

namespace: **default**

`spec`:

ports:

- port: 8081

protocol: TCP

targetPort: 8081

selector:

app: k8s-applicazione-2

sessionAffinity: None

`type`: LoadBalancer

- **ports** — su quale porta viene esposta l'applicazione.
- **selector** — a quale deployment (cioè a quali pod) fa riferimento il servizio. Di nuovo un match sulle label.
- **sessionAffinity: None** — se in una web application tutte le richieste di una sessione debbano andare **alla stessa istanza**. Qui sono servizi REST, quindi non interessa.

- **type** — il tipo di servizio.

Deployare una vera applicazione Spring Boot

Lo scenario. Due applicazioni:

- **applicazione 1** — un `@RestController` con un metodo su `/api/ciao` che risponde `ciao mamma`;
- **applicazione 2** — usa un **client Feign** per invocare l'applicazione 1. Espone `/provaws/helloworld`.

Il punto concettuale della lezione. ★ Si vogliono **eliminare Eureka, Config Server e Gateway**, perché **quei servizi li offre già Kubernetes**. Si tolgono tutti i riferimenti dall'applicazione e si usano direttamente i servizi del cluster.

Spring Cloud	Equivalente in Kubernetes
Eureka (service discovery)	Service + DNS interno del cluster
Ribbon / load balancer	Service LoadBalancer / ClusterIP
Spring Cloud Gateway	Service LoadBalancer , o Ingress
Config Server	ConfigMap e Secret , o variabili d'ambiente

Far vedere quale istanza risponde. Siccome aumenterà le istanze, il controller viene modificato per restituire **anche host e porta**:

```
@RestController
@RequestMapping("/provaws")
public class ProvaController {

    @Autowired private Environment environment;
    @Autowired private Applicazione1Client client;

    @GetMapping("/helloworld")
    public String helloWorld() {
        String msg = client.ciao();           // chiama applicazione 1
        String host = environment.getProperty("HOSTNAME");
        String port = environment.getProperty("local.server.port");
        return host + " - " + port + " - msg ricevuto: " + msg;
    }
}
```

`Environment` si ottiene **tramite dependency injection**.

Il pezzo che rompe tutto. Nel client Feign l'URL dell'applicazione 1 non si può scrivere a mano, perché l'indirizzo vero non lo conosci. Si usa una **variabile d'ambiente creata automaticamente da Kubernetes** per ogni Service:

```
@FeignClient(name = "applicazione-1",
url = "http://${K8S_APPLICAZIONE_1_SERVICE_HOST}:8081")
public interface Applicazione1Client {
    @GetMapping("/api/ciao")
    String ciao();
}
```

★ **L'errore che il docente racconta di aver pagato caro** («dopo aver bestemmiato in *N lingue*»): l'app si chiama **k8s-applicazione-1 con i trattini**, ma quando **Kubernetes crea le variabili d'ambiente i trattini diventano underscore**. Va scritto **K8S_APPLICAZIONE_1_SERVICE_HOST**. Con il trattino non funziona.

Regola generale: per un Service **nome-servizio** esistono **NOME_SERVIZIO_SERVICE_HOST** e **NOME_SERVIZIO_SERVICE_PORT**.

Alternativa più pulita che il corso non usa: il **DNS interno**, cioè **http://k8s-applicazione-1:8081** — con i trattini e senza variabili. Funziona anche se il Service viene creato *dopo* il pod, mentre le variabili d'ambiente vengono iniettate solo all'avvio del pod.

Le immagini. Per ciascuna app: plugin Docker nel `pom.xml` (con user, password ed email dell'account Docker Hub), `mvn clean install` per il jar, **avviare Docker Desktop** (il docente se ne dimentica e non funziona nulla), poi il goal Docker che crea l'immagine e la pusha sull'hub.

Il deploy:

```
kubectl create deployment k8s-applicazione-1 --image=<utente>/k8s-applicazione-1:0.0.1-SNAPSHOT
```

```
kubectl expose deployment k8s-applicazione-1 --type=LoadBalancer --port=8080
```

```
kubectl create deployment k8s-applicazione-2 --image=<utente>/k8s-applicazione-2:0.0.1-SNAPSHOT
```

```
kubectl expose deployment k8s-applicazione-2 --type=LoadBalancer --port=8081
```

Verifica: `<ip>:8080/api/ciao → ciao mamma; <ip>:8081/provaws/helloworld`
→ host + porta + messaggio ricevuto.

Il risultato che conta: l'endpoint esterno **rimane fisso** qualunque sia il numero di pod dietro. Nell'esempio c'è 1/1, ma potrebbero essere tre, quattro, cinque.

Deployare con kubectl apply

Perché i file YAML invece dei comandi. Tutto si può fare da prompt o graficamente dalla console, ma nella realtà si scrivono i YAML per tre motivi:

1. sono **file pronti per qualsiasi server compatibile con Kubernetes** — Amazon, Azure, on-premise — non solo per Google Cloud;
2. si evita di far scrivere allo sviluppatore **tutti i comandi** per duplicare e configurare le applicazioni;
3. *«devo replicare la mia applicazione su un altro Kubernetes: giro direttamente il mio YAML, già bello configurato, senza impazzire.»*

In una parola: **il YAML è riproducibile e versionabile, i comandi no.**

Parte A — modificare una risorsa esistente.

```
kubectl get deployment k8s-applicazione-1 -o yaml
```

Si copia in un file di testo, si cambia `replicas: 1` in `replicas: 2`, e — siccome il file è in locale ma si lavora sulla Cloud Shell — si apre l'**editor di Google Cloud**, `File` → `New File`, si incolla e si salva.

Il primo salvataggio **fallisce per un problema di permessi**: va salvato dentro la propria **home** (`/home/<utente>`).

```
kubectl apply -f k8s-applicazione-1-mod.yaml
```

```
kubectl get pod
```

Compare **un nuovo pod**: adesso ce ne sono due, uno creato da 39 secondi. Il **Warning** all'`apply` è normale — è l'avviso su `last-applied-configuration`, perché la risorsa era stata creata con `create/expose` e non con `apply`.

Parte B — deploy completo da zero.

```
kubectl get deployment -o wide
```

```
kubectl delete deployment k8s-applicazione-2
```

Kubernetes elenca cosa sta eliminando — **pod, service, deployment** — e restituisce il prompt a fine lavoro. `kubectl get all` conferma che non c'è più niente. Il docente sottolinea che usa i comandi invece della UI perché sono **standard e validi per tutti i provider**.

Poi carica il file del deployment già ripulito e lo applica:

```
kubectl apply -f k8s-applicazione-2.yaml
```

```
kubectl get all
```

Il deployment e i pod ci sono. **Ma non sono esposti**: *«abbiamo creato i vari pod ma non li abbiamo esposti»*. Serve il secondo file:

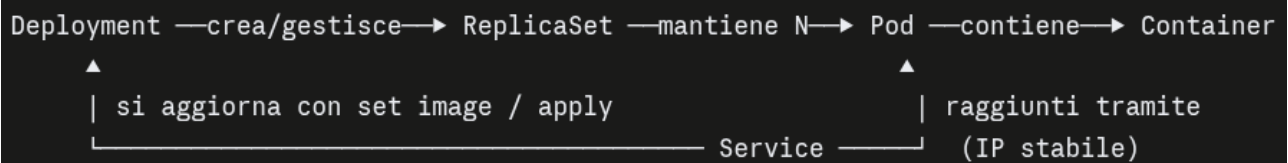
```
kubectl apply -f k8s-applicazione-2-service.yaml
```

E finalmente `http://<ip>:8081/provaws/helloworld` risponde: *«la mamma continua a salutarci»*.

★ **Il punto da non dimenticare: servono due file, uno per il Deployment e uno per il Service.** Applicare solo il deployment crea i pod ma non li rende raggiungibili. È la causa più comune di *"ho fatto il deploy ma non risponde"*.

Nota utile non del corso: più YAML possono stare in un unico file separati da `---`, o in una cartella da applicare in blocco con `kubectl apply -f cartella/`. E `--dry-run=client` verifica la sintassi prima di applicare davvero.

Quadro d'insieme



Comando	Cosa fa
<code>kubectl create deployment <n> --image=</code>	crea il deployment (e con esso ReplicaSet e pod)
<code>kubectl expose deployment <n> --type=LoadBalancer --port=P</code>	crea il Service e lo espone all'esterno
<code>kubectl get pod -o wide</code>	pod, nodo, IP, stato
<code>kubectl describe pod <id></code>	dettaglio, nome del container, Events con la causa degli errori
<code>kubectl get rs</code>	ReplicaSet: desiderati / correnti / pronti
<code>kubectl scale deployment <n> --replicas=N</code>	scala l'applicazione
<code>kubectl set image deployment/<n> <container>=:<tag></code>	aggiorna la versione
<code>kubectl get <risorsa> <n> -o yaml</code>	estrae il YAML da ripulire
<code>kubectl apply -f <file>.yaml</code>	crea o aggiorna da file
<code>kubectl delete deployment <n></code>	elimina deployment, replicaset e pod
<code>kubectl get all</code>	tutto insieme
Le tre trappole che risolvono l'80% dei problemi: le label devono combaciare tra selector , template e Service ; il Service va creato (non basta il deployment); nelle variabili d'ambiente dei Service i trattini diventano underscore .	